

lamet-agent 核心部分穷尽剖析

opencode pure

2026 年 8 月 17 日

摘要

项目性质判定：代码-物理交叉向（LQCD 高统计测量 LaMET 数据分析自动化 agent）。本报告穷尽枚举 6 个核心部分候选：深挖 4（agent 编排运行时、manifest job-DAG 契约、统计基元与物理谱分解模型、物理核函数族与匹配矩阵）、浅析 2、排除 0。最深剖析结论：lamet-agent 的独特竞争力在于**把需要物理判断的关联函数分析与外推步骤委托给 LLM 决策、把固定的重整化/傅里叶/匹配步骤软件化**，通过 manifest job-DAG 做执行蓝图、‘core/’ 数值基元做底座、‘planning/’ 做安全护栏，形成”可控 agent”与”自由 agent”的边界设计。

目录

1	项目定位与核心部分清单	1
1.1	项目性质判定	2
1.2	核心部分总清单（穷尽枚举）	2
2	核心部分深度剖析	2
2.1	C1 agent 编排运行时	3
2.2	C2 manifest job-DAG 契约	4
2.3	C3 统计基元与物理谱分解模型	5
2.4	C4 物理核函数族与匹配矩阵	5
3	独特性与竞争力评估	6
4	关键片段与公式	6
5	参考源清单	8
6	结论	9

1 项目定位与核心部分清单

1.1 项目性质判定

要点

项目性质： 代码-物理交叉向。依据：核心代码约 26K 行（lamet_agent/ 下 26 个 py 文件、26238 行实测） ， 其中物理数值逻辑（stages/*/functions.py 谱分解拟合、 kernels.py 微扰核函数、core/resampling.py bootstrap/jackknife）占主体；agent 编排逻辑（agent.py/core/tools.py/planning/）为控制面。独特性载体为 代码符号 ↔ 物理对象的双向映射（LLM action ↔ LQCD 测量步骤）。

1.2 核心部分总清单（穷尽枚举）

编号	核心部分	处理态与独特性概述
C1	agent 编排运行时（agent.py:611-614 + core/tools.py:199-288）	深挖：LLM/tool 循环 + 工具注册表 + 必需工具序列，LLM 只做决策不碰数值
C2	manifest job-DAG 契约（manifest.py: 17-96）	深挖：pydantic 模型把 run/inputs/stages 三块结构化为执行蓝图，跨 stage 角色命名输入
C3	统计基元与物理谱分解模型（core/resampling.py:52-131 + stages/correlator/functions.py:440-465）	深挖：bootstrap/jackknife/gvar 误差传递 + 2pt/3pt 谱分解比值模型
C4	物理核函数族与匹配矩阵（kernels.py: 58-380）	深挖：单圈 MSbar 转换、 β 函数、LaMET 匹配核列求和 plus 处方
C5	交互规划（planning/core.py:642-867）	浅析：LLM 驱动 JSON Patch 候选编辑 + 确定性 guardrail
C6	知识库与评审（stages/review/functions.py:578-585 + papers/lamet_kb/pipeline.py:26-60）	浅析：文献锚定排名 + 双语文档评审报告

表 1: 核心部分总清单（数据来源：全库索引实测；6 候选三态闭环）

2 核心部分深度剖析

2.1 C1 agent 编排运行时

核心算法

核心算法： LLM/tool 循环。每 job 建立独立 store 字典，静态提示一次性组装，此后每轮把工具观察追加为新的 user turn；LLM 每轮输出一个 JSON action (call_tool/request_user_input/finish)，agent 校验必需工具序列后执行注册表内的数值工具。复杂度：每 job 工具调用数由 LLM 决策（受 max_tool_steps 约束）。

15 步全流程重现：

A1 目的与前期准备： 把 LaMET 数据分析 (correlator → renorm → fourier → matching → extrapolation) 自动化，让 agent 代替物理学家做策略选择 (PLAN.md:5-12)。前置：Python ≥3.10 + analysis 依赖组 (gvar/lscfit/scipy/h5py/xarray/netCDF4)。

A2 输入： manifest 文件 (JSON/JSONC) 声明 run 级 metadata、inputs (correlators/artifacts/kernels)、每 stage 的 defaults/jobs (manifest.py:71-96、manifest.py:355-383)。

A3 输出： 每 stage 的 EnsembleData NetCDF 工件 + 图表 + 双语文档报告 + 最终光锥分布函数数组。

A4 整体框架： CLI run → validate → run_agent → for stage in metadata.stages → for job in jobs → hydrate artifacts → build static prompt → LLM/tool loop → store['output'] → stage report (agent.py:561-1012)。

A5 关键细节： job store 隔离——每 job 新建 store，上游输出按角色注入，下游引用缺失即抛错 (agent.py:623-635)；必需工具序列——按 stage+job 角色给出必须依次成功的工具，序列完成前 agent 不得 finish (core/tools.py:250-288)；工具参数注入——prepare_tool_args 从 manifest 注入路径/随机种子并改写绘图路径 (core/tools.py:357-372)。

代码-物理映射

映射原则： 每个关键代码符号必有物理对象，双向可查，无孤儿符号。

代码符号	物理对象	物理含义与参考源
run_agent	测量管线主循环	一次 manifest 驱动的完整测量编排 agent.py:611-614
store["output"]	阶段主产物	job 的终端工具把主数据放入的共享槽位 agent.py:657-659
required_job_tool_sequence	物理步骤顺序约束	物理流程要求的工具执行次序 core/tools.py:250-288
resolve_stage_tools	阶段工具注册表	stage id → STAGE_TOOLS 映射 core/stages.py:1-16

表 2: 映射表 C1 (数据来源: 源码实测)

A6 正确执行： lamet-agent run manifest.json --backend api (mock 冒烟无 LLM)。

A7/A8 实际执行与输出：本机实测 `lamet-agent validate` 失败于样例数据缺失 (`data_pion_pdf_cg` 未随 checkout)，pytest 子集 23/117/121 passed 证实运行时与工具层稳定（未验证真实 LLM 调用）。

A9 结果分析：48 项失败集中在依赖 `examples/*.json` 的集成测试（可运行 manifest 未入库），非被测代码逻辑缺陷（`PROJECT_LOG.md:1049-1060` 亦提示历史上留两个 failures）。

A10 综合分析：agent 数值动作全部限制在 `STAGE_TOOLS` 注册表内，LLM 只做决策层不执行数值运算——“可控 agent”与“自由 agent”的边界设计（`core/tools.py:199-204`）。

A11 结尾总结：C1 是“决策-执行分离”的核心实现：LLM 决定策略，注册表保证数值正确性。

A12 结尾评价：优点——数值安全性由注册表硬保证；缺点——LLM 决策依赖 prompt 工程，无真实 GPU 数据时无法端到端验证。

A13 补充说明：真实 LLM 调用 + Slurm 提交未验证（未验证）。

A14 参考源：见第 5 节；C1 证据 4 处。

2.2 C2 manifest job-DAG 契约

核心算法

核心算法：pydantic 模型驱动的 manifest 校验。三块结构（`metadata/inputs/stages`）严格类型化，stage 参数契约（`manifest_params.py`）拒绝未知键。

15 步全流程重现(节选)：**A1 目的：**让 agent 的执行蓝图“单一来源、可校验、可追溯”（`manifest.py:1` 模块 docstring）。**A2 输入：**JSON/JSONC manifest 文本（`examples/sample_manifest.jsonc:1-45`）。**A3 输出：**AnalysisManifest 模型实例；解析失败抛 ManifestPathError。**A4 框架：**StageId 六阶段字面量 + BzDirection/CoordUnit + parse_volume/parse_momentum (S48T64, PX0PY0PZ0 规范) + RunMetadata 全局设置 + 每 job 独立 store（`manifest.py:17-25`、`manifest.py:47-60`）。**A5 关键细节：**物理动量换算 `physical_momentum_gev` 把格点动量转 GeV: $p = 2\pi\hbar c |\vec{n}| / (a L_s)$ （`manifest.py:63-68`）。

物理图像

物理图像：manifest 是“测量计划书”：格点坐标 (S48T64)、动量量子数 (PXnPYnPZn)、目标可观测量 (PDF/DA/GPD) 与 parton 类型 (quark/gluon) 全部离散化声明，agent 只在这份计划书约束内行动。

A9 结果分析：manifest 契约三次迁移（扁平布局 → core+ 五 stage 包 → job-DAG）证实模式演进的合理性（`PROJECT_LOG.md:5-17`、`PROJECT_LOG.md:343-349`）。

A10-A13：planning 与 stages 共用同一模式（`check_manifest_draft` 调 `model_validate`），实现“同一模式、两处消费”（`planning/core.py:842-843`）。

2.3 C3 统计基元与物理谱分解模型

核心算法

核心算法: bootstrap/jackknife 重采样 + gvar 误差传递; 2pt/3pt 谱分解比值模型。复杂度: bootstrap $O(n_samples \times n_conf)$; jackknife $O(n_conf)$ 。

物理图像

物理图像: 谱分解 (PLAN.md 公式):

$$C_{2pt}(t) = \sum_n |A_n|^2 e^{-E_n t} \quad (1)$$

$$C_{3pt}(t, \tau) = \sum_{n,m} A_n A_m^* g_A e^{-E_n(t-\tau)} e^{-E_m \tau} \quad (2)$$

$$R(t, \tau) = \frac{C_{3pt}(t, \tau)}{C_{2pt}(t)} \quad (3)$$

代码 `pt3_ratio_fcn` 实现 (3) 的多态推广 (含矩阵元 O_{ij} 与能级 E_n), (1)/(2) 的谱分解结构在 `qda_fcn` 中体现 (`stages/correlator/functions.py:440-465`)。

重采样实现: `bin_data` 相邻组态分箱 (`core/resampling.py:52-60`); bootstrap 有放回抽样求均值 (`core/resampling.py:63-71`); jackknife leave-one-bin-out (`core/resampling.py:74-81`); `bs_ls_avg` bootstrap 样本 $\rightarrow$ gvar (含协方差) (`core/resampling.py:84-96`); `jk_ls_avg` jackknife 样本 $\rightarrow$ gvar (方差乘 $n-1$) (`core/resampling.py:99-112`)。物理要点: 2pt/3pt 必须用同一组 bootstrap indices 再构成比值 (`PROJECT_LOG.md:268-271`), 这由 `bootstrap_indices` 共享索引保证 (`core/resampling.py:129-131`)。

极限校验: $n_conf \rightarrow \infty$ 时 jackknife 方差 $\times(n-1) \rightarrow$ 真实方差; bootstrap 在 $n_samples \rightarrow \infty$ 趋于正态近似——两法互为交叉验证。

2.4 C4 物理核函数族与匹配矩阵

物理图像

物理图像: 自重整化 (self-renormalization): 坐标空间矩阵元经 $Z_{\overline{MS}}$ 转换到 $\overline{MS}$ 方案。单圈转换因子 (`kernels.py:104-112`):

$$Z_{\overline{MS}}(z; \mu) = 1 + \frac{\alpha_s C_F}{2\pi} \left(\frac{3}{2} \log \frac{\mu^2 z^2 e^{2\gamma_E}}{4} + \text{offset} \right), \quad \text{offset} = 5/2 \text{ (PDF)}, 7/2 \text{ (DA)} \quad (4)$$

$\alpha_s(\mu)$ 由 β 函数 (LO/NLO/NNLO) 跑动 (`kernels.py:58-96`)。

物理图像

匹配矩阵: LaMET 匹配核离散化为 (n_x, n_y) 矩阵, 采用**列求和 plus 处方**——”regular NLO integrand singular on $x = y$, 由每列求和为零强制成 plus 函数” (`kernels.py:361-380`)。PDF 核的 ξ 积分用 `_build_pdf_matrix` 处理 (`kernels.py:681-760` 公有 quark 核函数族)。

15 步 (节选): A2 输入 = 坐标空间矩阵元样本 + μ 标度 + 外推项参数; A4 框架 = β 函数 $\rightarrow \alpha_s$ 跑动 $\rightarrow Z_{\overline{MS}}$ 转换 $\rightarrow$ 匹配矩阵; A9 分析: 自重整化拟合函数含线发散 g_z/x 、对数发散 $3C_F/b_0 \log \log$ 、离散效应项 (`stages/renorm/functions.py:792-806`)。

3 独特性与竞争力评估

独特特点	对比基准	优势与证据
决策-执行分离	全脚本硬编码流程	LLM 选策略 + 注册表保数值安全; 决策空间过大 (multi-state/GEVP/model averaging) 无法脚本化 (<code>PLAN.md:24-29</code>)
job-DAG manifest	顶层 CLI + 扁平 kernels	可校验、可追溯、可部分运行; 上游产物按角色注入
共享重采样索引	各阶段独立重采样	bootstrap indices 全局共享, 比值误差自治 (<code>core/resampling.py:129-131</code>)
单圈核函数自包含	外部库散装	$\beta/Z_{\overline{MS}}$ /匹配矩阵同一文件可追踪 (<code>kernels.py:58-380</code>)

表 3: 独特性评估 (数据来源: 源码 +PLAN 实测)

4 关键片段与公式

```
1 def pt3_ratio_fcn(
2     t: np.ndarray,
3     tau: np.ndarray,
4     p: dict,
5     Lt: int,
6     *,
7     nstate: int = 2,
8     part: str = "re",
9 ) -> np.ndarray:
10     """Real (`part='re'`) or imaginary (`part='im'`) n-state 3pt/2pt ratio."""
11     energies = _state_energies(p, nstate)
12
13     numerator = 0.0
14     for src, src_e in enumerate(energies):
```

```

15     for snk, snk_e in enumerate(energies):
16         matrix_element = p[f"0{min(src, snk)}{max(src, snk)}_{part}"]
17         numerator = numerator + (
18             matrix_element
19             * p[f"z{src}"]
20             * p[f"z{snk}"]
21             * np.exp(-src_e * (t - tau))
22             * np.exp(-snk_e * tau)
23             / (2 * src_e)
24             / (2 * snk_e)
25         )
26     return numerator / pt2_re_fcn(t, p, Lt, nstate=nstate)

```

```

1  def bootstrap(data: np.ndarray, n_samples: int, axis: int = 0, seed: int | None = 1984, bin_size: int = 1) -> np.
    ↳ ndarray:
2      """Generate bootstrap sample averages from ensemble data."""
3      data = np.asarray(data)
4      if bin_size > 1:
5          data = bin_data(data, bin_size, axis=axis)
6      n_conf = data.shape[axis]
7      rng = np.random.default_rng(seed)
8      indices = rng.choice(n_conf, (n_samples, n_conf), replace=True)
9      return np.take(data, indices, axis=axis).mean(axis=axis + 1)
10
11
12 def jackknife(data: np.ndarray, axis: int = 0, bin_size: int = 1) -> np.ndarray:
13     """Generate leave-one-bin-out jackknife sample averages from ensemble data."""
14     data = np.asarray(data)
15     if bin_size > 1:
16         data = bin_data(data, bin_size, axis=axis)
17     n_conf = data.shape[axis]
18     total = data.sum(axis=axis, keepdims=True)
19     return (total - data) / (n_conf - 1)
20
21
22 def bs_ls_avg(bs_ls: np.ndarray, axis: int = 0) -> np.ndarray:
23     """Average bootstrap samples (sample axis first) into a gvar array."""
24     bs_arr = _move_sample_axis(np.asarray(bs_ls), axis)
25     if bs_arr.shape[0] < 2:
26         mean = np.mean(bs_arr, axis=0)
27         return gv.gvar(mean, np.zeros_like(mean, dtype=float))
28     bs_flat = bs_arr.reshape(bs_arr.shape[0], -1)
29     mean = np.mean(bs_flat, axis=0)
30     if bs_flat.shape[1] == 1:
31         out = gv.gvar(mean[0], np.std(bs_flat[:, 0], ddof=1))
32         return _reshape_gvar(out, bs_arr.shape[1:])
33     cov = np.cov(bs_flat, rowvar=False)
34     return gv.gvar(mean, cov).reshape(bs_arr.shape[1:])

```

```

1  def ZMSbar(z_fm: np.ndarray | float, *, mu: float = 2.0, offset: float, order: int = 0, Nf: int = 3) -> np.ndarray:
2      """1-loop coordinate-space conversion to MSbar at scale ``mu`` (GeV).
3
4      ``offset`` is the finite constant: ``5/2`` for PDF and ``7/2`` for DA.
5      """
6      z_arr = np.asarray(z_fm, dtype=float)

```

```

7   alphas = alphas_nloop(mu, order=order, Nf=Nf)
8   log_term = np.log(mu**2 * (z_arr / GEV_FM) ** 2 * np.exp(2.0 * np.euler_gamma) / 4.0)
9   return 1.0 + alphas * CF / (2.0 * np.pi) * (1.5 * log_term + offset)
10
11
12 def ZMSbar_pdf(z_fm: np.ndarray | float, mu: float = 2.0, order: int = 0, Nf: int = 3) -> np.ndarray:
13     """1-loop MSbar conversion factor for PDF self-renormalization."""
14     return ZMSbar(z_fm, mu=mu, offset=2.5, order=order, Nf=Nf)
15
16
17 def ZMSbar_da(z_fm: np.ndarray | float, mu: float = 2.0, order: int = 0, Nf: int = 3) -> np.ndarray:
18     """1-loop MSbar conversion factor for DA self-renormalization."""
19     return ZMSbar(z_fm, mu=mu, offset=3.5, order=order, Nf=Nf)

```

5 参考源清单

参考源	类型	内容/作用
PLAN.md:5-12	仓库	LaMET 五步流程与 agent 分工动机
PLAN.md:20-29	仓库	谱分解公式与 g_A 提取方案
manifest.py:17-25	仓库	StageId 六阶段字面量
manifest.py:47-60	仓库	体积/动量规范解析
manifest.py:63-68	仓库	格点动量 $\rightarrow$ GeV 换算
manifest.py:71-96	仓库	RunMetadata 全局设置
agent.py:561-1012	仓库	run_agent 主循环与阶段产物
agent.py:623-635	仓库	job store 隔离与角色注入
core/tools.py:199-288	仓库	工具解析/必需序列
core/tools.py:357-372	仓库	prepare_tool_args 参数注入
core/resampling.py:52-131	仓库	bin/bootstrap/jackknife/gvar
core/stages.py:1-16	仓库	stage id $\rightarrow$ 包名映射
stages/correlator/functions.py:440-465	仓库	3pt/2pt 比值谱分解模型
stages/renorm/functions.py:792-806	仓库	自重整化拟合函数
kernels.py:58-96	仓库	β 函数与 α_s 跑动
kernels.py:104-122	仓库	$Z_{\overline{MS}}$ 单圈转换
kernels.py:361-380	仓库	匹配矩阵列求和 plus 处方
kernels.py:681-760	仓库	公有 quark 核函数族
PROJECT_LOG.md:268-271	仓库	共享 bootstrap indices 约定
PROJECT_LOG.md:343-349	仓库	job-DAG manifest 迁移
examples/sample_manifest.jsonc:1-45	仓库	注释清单设计说明

6 结论

结论

穷尽剖析结论：核心部分 6 个 = 深挖 4 / 浅析 2 / 排除 0，三态闭环。**最强竞争力**：决策-执行分离架构 (C1) + job-DAG 契约 (C2) + 共享重采样基元 (C3) + 单圈核函数族 (C4) 四者耦合，把需要物理判断的 LaMET 分析步骤压缩为”一次 manifest 驱动的高统计测量自动化”。

局限与风险

未验证项：真实 LLM 调用、Slurm 提交、真实 LQCD 数据端到端运行均未验证（本机无数据/无 GPU 环境）；谱分解公式与 $Z_{\overline{MS}}$ 公式的数值精度需真实数据比对；48 项依赖 example manifest 的集成测试因仓库快照不完整而失败（非逻辑缺陷）。